

UNIVERSITEIT VAN PRETORIA
UNIVERSITY OF PRETORIA
YUNIBESITHI YA PRETORIA
Denkleiers • Leading Minds • Dikgopolo tša Dihlalefi

Department of Computer Science
Faculty of Engineering, Built Environment & IT
University of Pretoria

Program design: Introduction

Copyright ©2026 by the 2026 staff of COS 110. All rights reserved.

Assignment 2 - Terminal Computer Graphics

Release Date: 07-09-2026 at 06:00

FitchFork Release Date: 14-09-2026 at 06:00

Due Date: 09-10-2026 at 23:59

Late Deadline: 10-10-2026 at 00:59

Read the entire specification before starting with the practical.

Contents

1 General Instructions **6**

2 Overview **7**

3 Your Task: **7**

3.1 Exception Hierarchy . . . 7

3.2 Vector . . . 8

3.2.1 Members . . . 8

3.2.1.1 `n: int` . . . 8

3.2.1.2 `arr: float*` . . . 8

3.2.2 Functions . . . 8

3.2.2.1 `Vector(int)` . . . 8

3.2.2.2 `Vector(int, float*)` . . . 8

3.2.2.3 `~Vector()` . . . 8

3.2.2.4 `Vector(const Vector&)` . . . 8

3.2.2.5 `operator+(const Vector&): Vector` . . . 8

3.2.2.6 `operator-(const Vector&): Vector` . . . 8

3.2.2.7 `operator*(float): Vector` . . . 9

3.2.2.8 `operator*(const Vector&): Vector` . . . 9

3.2.2.9 `operator[](int): const float&` and `operator[](int): float&` . . . 9

3.2.2.10 `magnitude(): float` . . . 9

3.2.2.11 `operator Matrix()` . . . 9

3.2.2.12 `crossProduct(const Vector&): Vector` . . . 9

3.2.2.13 `unitVector(): Vector` . . . 9

3.2.2.14 `getN(): int` . . . 9

3.2.2.15 `operator>(const Vector&): bool` . . . 10

3.2.2.16 `operator~(): Vector` . . . 10

3.2.2.17 `operator<<(ostream&, const Vector&): ostream&` . . . 10

3.3 Matrix . . . 10

3.3.1 Members . . . 10

3.3.1.1 `n: int` . . . 10

3.3.1.2 `m: int` . . . 10

3.3.1.3 `arr: Vector**` . . . 10

3.3.2 Functions . . . 11

3.3.2.1 `Matrix(int n, int m)` . . . 11

3.3.2.2 `Matrix(int n, int m, Vector**)` . . . 11

3.3.2.3 `Matrix(const Matrix&)` . . . 11

3.3.2.4 `~Matrix()` . . . 11

3.3.2.5 `operator[](int): Vector&` . . . 11

3.3.2.6 `operator*(const Matrix&): Matrix` . . . 11

3.3.2.7 `operator*(const Vector&): Vector` . . . 11

3.3.2.8 operator*(const float): Matrix . . . 11
3.3.2.9 operator+(const Matrix&): Matrix . . . 12
3.3.2.10 operator∼(): Matrix . . . 12
3.3.2.11 getM(): int and getN(): int . . . 12
3.3.2.12 operator<<(ostream&, const Matrix&): ostream& . . . 12
3.4 SquareMatrix . . . 12
3.4.1 Functions . . . 12
3.4.1.1 SqaureMatrix(int n) . . . 12
3.4.1.2 SquareMatrix(int n, Vector** arr . . . 12
3.4.1.3 SquareMatrix(const SquareMatrix&) . . . 12
3.5 TransformationMatrix . . . 13
3.5.1 Functions . . . 13
3.5.1.1 TransformationMatrix() . . . 13
3.5.1.2 TransformationMatrix(const TransformationMatrix&) . . . 13
3.5.1.3 translate(Vector& dir): TransformationMatrix . . . 13
3.5.1.4 scale(Vector& dir): TransformationMatrix . . . 13
3.5.1.5 rotate(float rad): TransformationMatrix . . . 14
3.5.1.6 shear(Vector& dir): TransformationMatrix . . . 14
3.5.1.7 reflect(): TransformationMatrix . . . 14
3.6 VertexArray . . . 14
3.6.1 Members . . . 15
3.6.1.1 size: int . . . 15
3.6.1.2 array: Vector** . . . 15
3.6.2 Functions . . . 15
3.6.2.1 VertexArray() . . . 15
3.6.2.2 VertexArray(const VertexArray&) . . . 15
3.6.2.3 ∼VertexArray() . . . 15
3.6.2.4 operator=(const VertexArray&): VertexArray& . . . 15
3.6.2.5 operator+=(const Vector&): VertexArray& . . . 15
3.6.2.6 operator+=(const VertexArray&): VertexArray& . . . 15
3.6.2.7 operator[](int): Vector& and operator[](int): const Vector& . . . 16
3.6.2.8 operator int() . . . 16
3.6.2.9 operator<<(ostream&, const VertexArray&): ostream& . . . 16
3.7 Shape . . . 16
3.7.1 Functions . . . 16
3.7.1.1 ∼Shape() . . . 16
3.7.1.2 operator=(const Shape&): Shape& . . . 16
3.7.1.3 getArray(): VertexArray . . . 16
3.7.1.4 applyMatrix(const TransformationMatrix&): Shape* . . . 16
3.7.1.5 clone(): Shape* . . . 16
3.7.1.6 formVertices(): VertexArray . . . 17
3.7.1.7 operator+=(const Shape&): ComboShape* . . . 17

3.7.1.8 operator+=(const ComboShape*): ComboShape* . . . 17
3.8 ComboShape . . . 17
3.8.1 Members . . . 17
3.8.1.1 numSubShapes: int . . . 17
3.8.1.2 subShapes: Shape** . . . 17
3.8.2 Functions . . . 17
3.8.2.1 ComboShape() . . . 17
3.8.2.2 ComboShape(const ComboShape&) . . . 17
3.8.2.3 ~ComboShape() . . . 17
3.8.2.4 operator=(const ComboShape&): ComboShape& . . . 17
3.8.2.5 getArray(): VertexArray . . . 18
3.8.2.6 applyMatrix(const TransformationMatrix&): ComboShape* . . . 18
3.8.2.7 formVertices(): VertexArray . . . 18
3.8.2.8 operator+=(const Shape&): ComboShape* . . . 18
3.8.2.9 operator+=(const ComboShape*): ComboShape* . . . 18
3.9 Point . . . 18
3.9.1 Members . . . 18
3.9.1.1 point: Vector . . . 18
3.9.2 Functions . . . 18
3.9.2.1 Point(const Vector&) . . . 18
3.9.2.2 Point(const Point&) . . . 19
3.9.2.3 operator=(const Point&): Point& . . . 19
3.9.2.4 getArray(): VertexArray . . . 19
3.9.2.5 applyMatrix(const TransformationMatrix&): Point* . . . 19
3.9.2.6 operator+=(const Shape&): ComboShape* and operator+=(const ComboShape*): ComboShape* . . . 19
3.9.2.7 getVector(): const Vector& . . . 19
3.10 Triangle . . . 19
3.10.1 Members . . . 19
3.10.1.1 p0: Point, p1: Point, and p2: Point . . . 19
3.10.2 Functions . . . 19
3.10.2.1 Triangle(const Vector& p0, const Vector& p1, const Vector& p2) . . . 19
3.10.2.2 Triangle(const Triangle&) . . . 20
3.10.2.3 formVertices(): VertexArray . . . 20
3.10.2.4 applyMatrix(const TransformationMatrix&): Traingle* . . . 20
3.11 Rectangle . . . 20
3.11.1 Functions . . . 20
3.11.1.1 Rectangle(const Vector& lt, const Vector& tr, const Vector& br, const Vector& bl) . . . 20
3.11.1.2 Rectangle(const Rectangle&) . . . 21
3.11.1.3 formVertices(): VertexArray . . . 21
3.12 Screen . . . 21

3.12.1 Members . . . 21
3.12.1.1 pixels: char** . . . 21
3.12.1.2 height: int . . . 21
3.12.1.3 width: int . . . 21
3.12.2 Functions . . . 21
3.12.2.1 Screen(height: int, width: int) . . . 21
3.12.2.2 ~Screen() . . . 21
3.12.2.3 operator<<(ostream&, const Screen&): ostream& . . . 22
3.12.2.4 implicitLineEquation(const Vector& p0, const Vector& p1, const Vector& p): float . . . 22
3.12.2.5 draw(int x, int y): void . . . 22
3.12.2.6 draw(const Vector& p0, const Vector& p1): void . . . 22
3.12.2.7 draw(const VertexArray&): void . . . 23
3.12.2.8 clearScreen(): void . . . 23

4 Memory Management **23**

5 Testing **23**

6 Implementation Details **24**

7 Upload Checklist **24**

8 Submission **25**

A Midpoint Algorithm **26**

B Background **27**
B.1 Implicit Line Equation . . . 27
B.2 Computer Graphics Screen . . . 28
B.3 Mid Point Algorithm . . . 28

1 General Instructions

- ***Read the entire assignment thoroughly before you begin coding.***
- **This assignment should be completed individually.**
- **Every submission will be inspected with the help of dedicated plagiarism detection software.**
- Be ready to upload your assignment well before the deadline. There is a late deadline which is 1 hour after the initial deadline which has a penalty of 20% of your achieved mark. **No extensions will be granted.**
- If your code does not compile, you will be awarded a mark of 0. The output of your program will be primarily considered for marks, although internal structure may also be tested (eg. the presence/absence of certain functions or structure).
- Failure of your program to successfully exit will result in a mark of 0.
- Note that plagiarism is considered a very serious offence. Plagiarism will not be tolerated, and disciplinary action will be taken against offending students. Please refer to the University of Pretoria's plagiarism page at `https://portal.cs.up.ac.za/files/departmental-guide/`.
- Unless otherwise stated, the usage of non-C++98 features or additional libraries outside of those indicated in the assignment will **not** be allowed. Some of the appropriate files that you have submit will be overwritten during marking to ensure compliance to these requirements. **Please ensure you use C++98**
- All functions should be implemented in the corresponding `cpp` file. No inline implementation in the header file apart from the provided functions.
- The usage of ChatGPT and other AI-Related software to generate submitted code is strictly forbidden and will be considered as plagiarism.
- The use of this practical, in any form, by any company/business/organisation outside the University of Pretoria is strictly forbidden. This includes, but is not limited to, the use of the practical for discussion sessions, review sessions, marketing tools, or providing certain aspects as a free service.

2 Overview

In this assignment, you will gain experience with operator overloading, inheritance, and abstract classes and functions. To achieve this, you will be building a primitive terminal-based 2D graphics engine.

For the curious students, the background for this assignment can be found in Appendinx B.

3 Your Task:

You are required to implement all of the functions laid out in this section.

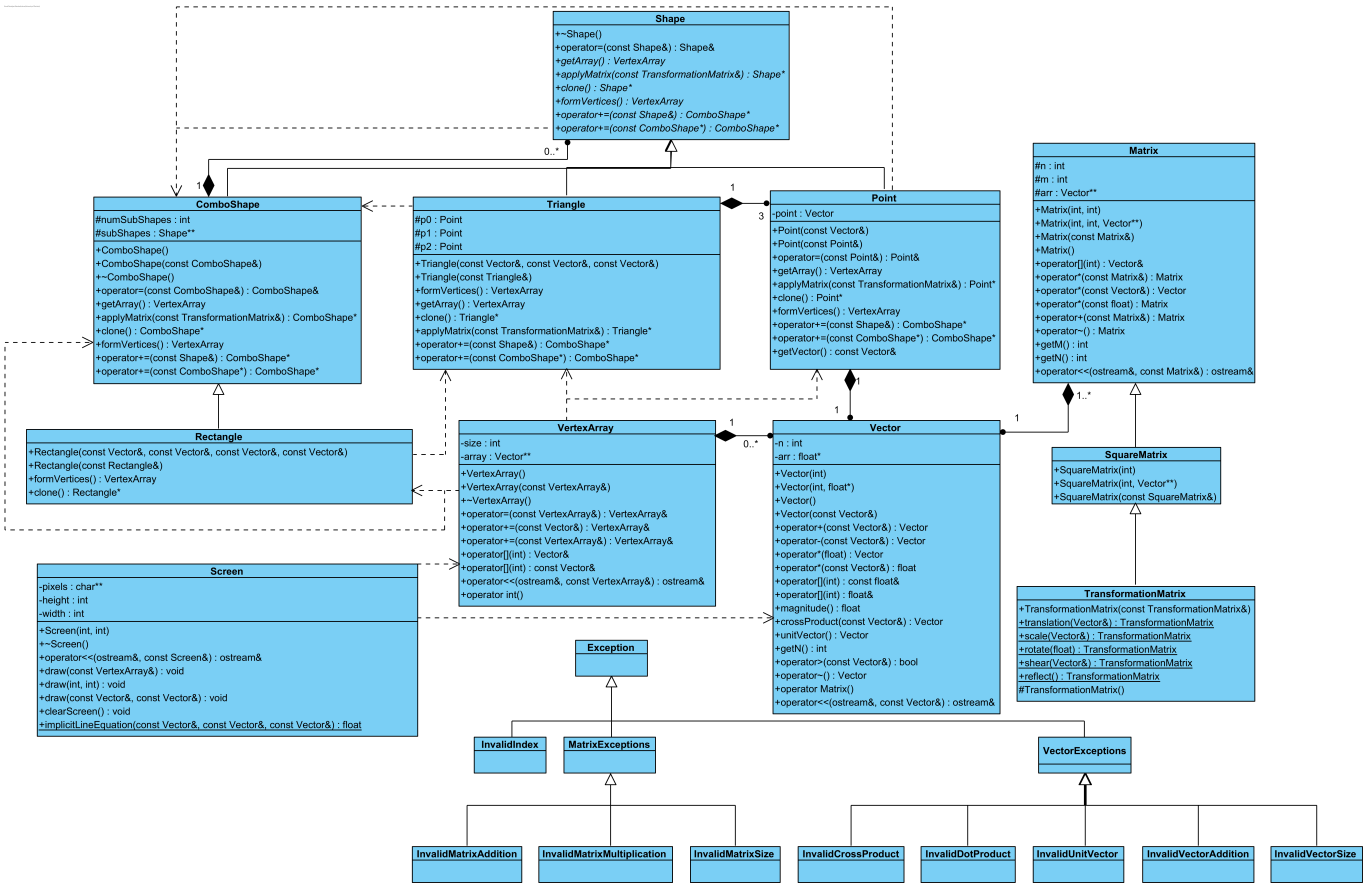


Figure 1: UML Class Diagram for the assignment

Please see the provided header files for **constness**, **virtualism**, and **friend** relations. Do not modify the provided header files, as they have been set up to compile in the correct order.

3.1 Exception Hierarchy

The `Exception` hierarchy is a set of provided exception classes that will be used throughout this assignment to elegantly handle problems that might occur. All the classes in the hierarchy can be found in the provided `Exception.h` file.

3.2 Vector

This class represents a vector in all possible N-D space, where N is a positive integer.

3.2.1 Members

3.2.1.1 `n: int`

- This is the dimensionality of the array

3.2.1.2 `arr: float*`

- This is the array of floats that makes up the vector.

3.2.2 Functions

3.2.2.1 `Vector(int)`

- This constructor receives the dimensionality of the vector as input.
- It should initialise all member variables accordingly and set each value in the array to 0.
- If the dimensionality is invalid, the constructor should throw an `InvalidVectorSize` object.

3.2.2.2 `Vector(int, float*)`

- This constructor receives the dimensionality of the vector, as well as values for the vector as input.
- It should initialise all member variables accordingly.
- It can be assumed that the inputs are valid.

3.2.2.3 `~Vector()`

- This is the destructor and should deallocate any dynamic memory allocated.

3.2.2.4 `Vector(const Vector&)`

- This is the copy constructor and should perform a deep copy of the passed-in object.

3.2.2.5 `operator+(const Vector&): Vector`

- This function should add the two vectors together and return the resulting vector.
- If the two vectors are not of the same dimensionality, throw an `InvalidVectorAddition` object.

3.2.2.6 `operator-(const Vector&): Vector`

- This function should subtract the two vectors and return the resulting vector.
- If the two vectors are not of the same dimensionality, throw an `InvalidVectorAddition` object.

3.2.2.7 `operator*(float): Vector`

- This function should scale the current vector by the passed-in amount and return the resulting vector.

3.2.2.8 `operator*(const Vector&): Vector`

- This function should perform member-wise multiplication of the two vectors and return the resulting vector.
- If the two vectors are not of the same dimensionality, throw an `InvalidVectorAddition` object.

3.2.2.9 `operator[](int): const float&` and `operator[](int): float&`

- These functions should return the float value at the passed-in index.
- If the index is invalid, the functions should throw an object of type `InvalidIndex`.

3.2.2.10 `magnitude(): float`

- This function should calculate the magnitude of the vector using the Euclidean norm.

3.2.2.11 `operator Matrix()`

- This is a casting operator[1] which converts the vector to a $n \times 1$ Matrix.

3.2.2.12 `crossProduct(const Vector&): Vector`

- This function should calculate the cross product between the two vectors.
- If either of the vectors has a dimensionality other than 3, then throw an object of type `InvalidCrossProduct`.

3.2.2.13 `unitVector(): Vector`

- This function should calculate the unit vector of the current vector and return the calculated vector.
- If the magnitude of the current vector is 0, throw an object of type `InvalidUnitVector`.

3.2.2.14 `getN(): int`

- This function should return the dimensionality of the vector.

[1]For more information on casting operators please visit: `https://en.cppreference.com/cpp/language/cast_operator`

3.2.2.15 operator>(const Vector&): bool

- This function should determine if one vector is greater than another based on the first element in the array.
- If the current vector is strictly greater than the passed-in vector, return true; otherwise return false.

3.2.2.16 operator~(): Vector

- This operator acts as a rounding operator, where each attribute in the vector is floored.
- Return this rounded vector.

3.2.2.17 operator<<(ostream&, const Vector&): ostream&

- This operator could populate the passed-in `ostream` object with the contents of the vector using the following format:

```
[arr[0],arr[1],...,arr[n]]$
```

  where `$` indicates a new line.

- Thus, consider the vector: $\begin{bmatrix} 5 \\ 3 \\ 1 \end{bmatrix}$ the printout should be:

```
[5,3,1]
```

3.3 Matrix

This class represents a row-major 2-D matrix of size $N \times M$.

3.3.1 Members

3.3.1.1 n: int

- This is the number of rows the matrix has.

3.3.1.2 m: int

- This is the number of columns the matrix has.

3.3.1.3 arr: Vector**

- This is a 1D dynamic array of dynamic `Vector` objects that represent each of the rows of the matrix.

3.3.2 Functions

3.3.2.1 `Matrix(int n, int m)`

- This constructor should initialise the appropriate members accordingly, including the array with vector objects.
- If either of the passed-in parameters is less than or equal to 0, throw an object of type `InvalidMatrixSize`.

3.3.2.2 `Matrix(int n, int m, Vector**)`

- This constructor should initialise the appropriate members accordingly, including a deep copy of the passed-in array.
- It can be assumed that all inputs are valid.

3.3.2.3 `Matrix(const Matrix&)`

- This copy constructor should perform a deep copy of the passed-in object.

3.3.2.4 `~Matrix()`

- This destructor should deallocate all dynamic memory associated with the object.

3.3.2.5 `operator[](int): Vector&`

- This function should return the vector object at the passed-in index.
- If the passed-in index is invalid, throw an object of type `InvalidMatrixMultiplication`.

3.3.2.6 `operator*(const Matrix&): Matrix`

- This function should return the product of the current matrix and the passed-in matrix.
- If the two matrices can't be multiplied, the function should throw an object of type `InvalidMatrixMulti`

3.3.2.7 `operator*(const Vector&): Vector`

- This function should return the product of the current matrix and the passed-in vector.
- *Hint: remember a vector can be considered as a special matrix, with specific dimensions.*
- If the matrix and the vector are incompatible for multiplication, throw an object of type `InvalidMatrixMultiplication`.

3.3.2.8 `operator*(const float): Matrix`

- This function should return a scaled copy of the current matrix using the passed-in parameter.

3.3.2.9 `operator+(const Matrix&): Matrix`

- This function should return the result of the sum of the current matrix and the passed-in matrix.
- If the two matrices are not of the same dimensionality, throw an object of type `InvalidMatrixAddition`.

3.3.2.10 `operator~(): Matrix`

- This function should return the transposed matrix of the current matrix.

3.3.2.11 `getM(): int` and `getN(): int`

- These functions should return the appropriate member variable, respectively.

3.3.2.12 `operator<<(ostream&, const Matrix&): ostream&`

- This function should populate the passed-in `ostream` object with the vectors contained in the array.
- *Hint: use the appropriate Vector function to populate the `ostream` object.*
- For example, consider the matrix: $\begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{bmatrix}$, the output should be:

```
[1,2,3]
[4,5,6]
```

3.4 SquareMatrix

A square matrix is a specialised type of matrix where the size is $N \times N$.

3.4.1 Functions

3.4.1.1 `SqaureMatrix(int n)`

- The constructor should initialise all member variables (including inherited) appropriately.

3.4.1.2 `SquareMatrix(int n, Vector** arr`

- The constructor should initialise all member variables (including inherited) appropriately.

3.4.1.3 `SquareMatrix(const SquareMatrix&)`

- The copy constructor should create a deep copy of the passed-in square matrix.

3.5 TransformationMatrix

In the case of this assignment, the transformation matrices are a specialised form of square matrices where each matrix is of size 3×3. These matrices are used to manipulate and transform the different shapes in a 2D graphics engine.

3.5.1 Functions

3.5.1.1 `TransformationMatrix()`

- This constructor should initialise the matrix to have the following values: $\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$

3.5.1.2 `TransformationMatrix(const TransformationMatrix&)`

- This copy constructor should create a deep copy of the passed-in transformation matrix.

3.5.1.3 `translate(Vector& dir): TransformationMatrix`

- This function creates a matrix that will translate (move) a shape around the screen.
- If the passed-in vector does not have a size of exactly 3, throw an object of type `InvalidVectorSize`.
- To achieve this, assume that the passed-in vector has the following values: $\begin{bmatrix} d_x \\ d_y \\ 1 \end{bmatrix}$, where d_x is the translation along the x-axis and d_y is the translation along the y-axis. The resulting matrix should thus have the form: $\begin{bmatrix} 1 & 0 & d_x \\ 0 & 1 & d_y \\ 0 & 0 & 1 \end{bmatrix}$
- For example, the matrix: $\begin{bmatrix} 1 & 0 & 2 \\ 0 & 1 & -3 \\ 0 & 0 & 1 \end{bmatrix}$, will move a shape left by 2 units and down by 3 units.

3.5.1.4 `scale(Vector& dir): TransformationMatrix`

- This function creates a matrix that will scale.
- If the passed-in vector does not have a size of exactly 3, throw an object of type `InvalidVectorSize`.
- To achieve this, assume that the passed-in vector has the following values: $\begin{bmatrix} s_x \\ s_y \\ 1 \end{bmatrix}$, where s_x is the scale factor along the x-axis and s_y is the scale factor along the y-axis. The resulting matrix should thus have the form: $\begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & 1 \end{bmatrix}$

- For example, the matrix: $\begin{bmatrix} 2 & 0 & 0 \\ 0 & 0.2 & 0 \\ 0 & 0 & 1 \end{bmatrix}$, will scale the shape by 2 units horizontally and by 0.2 units vertically. Thus, stretching the shape horizontally but squashing it vertically.

3.5.1.5 `rotate(float rad): TransformationMatrix`

- This function creates a matrix that will rotate a shape around the screen by some number of radians.
- Assume that the passed-in angle is θ, the resulting matrix should thus have the form: $\begin{bmatrix} \cos(\theta) & -\sin(\theta) & 0 \\ \sin(\theta) & \cos(\theta) & 0 \\ 0 & 0 & 1 \end{bmatrix}$
- For example, if rad $= \frac{\pi}{4}$, the matrix: $\begin{bmatrix} \cos(\frac{\pi}{4}) & -\sin(\frac{\pi}{4}) & 0 \\ \sin(\frac{\pi}{4}) & \cos(\frac{\pi}{4}) & 0 \\ 0 & 0 & 1 \end{bmatrix}$, will rotate a shape by 45° about the x-axis.

3.5.1.6 `shear(Vector& dir): TransformationMatrix`

- This function creates a matrix that will shear the shape.
- If the passed-in vector does not have a size of exactly 3, throw an object of type `InvalidVectorSize`.
- To achieve this, assume that the passed-in vector has the following values: $\begin{bmatrix} s_x \\ s_y \\ 1 \end{bmatrix}$, where s_x is the shear factor along the x-axis and s_y is the shear factor along the y-axis. The resulting matrix should thus have the form: $\begin{bmatrix} 1 & s_x & 0 \\ s_y & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$

3.5.1.7 `reflect(): TransformationMatrix`

- This function creates a matrix that will reflect the shape around the y-axis.
- To achieve this, the matrix should have the form: $\begin{bmatrix} -1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$

3.6 VertexArray

This is a wrapper class for an array of vertices, which will be used by the screen to draw different shapes.

3.6.1 Members

3.6.1.1 `size: int`

- This is the size of the array.

3.6.1.2 `array: Vector**`

- This is a 1D dynamic array of dynamic `Vector` objects.

3.6.2 Functions

3.6.2.1 `VertexArray()`

- This constructor should set both member variables to 0/`NULL`.

3.6.2.2 `VertexArray(const VertexArray&)`

- This copy constructor should create a deep copy of the passed-in object.

3.6.2.3 `~VertexArray()`

- The destructor should deallocate all dynamic memory associated with the object.

3.6.2.4 `operator=(const VertexArray&): VertexArray&`

- The assignment operator should assign the passed-in parameter to the current object as discussed in the lectures.
- A deep copy of each `Vector` object should be made.

3.6.2.5 `operator+=(const Vector&): VertexArray&`

- This operator should append the passed-in parameter to the array by resizing the array accordingly.
- Return the modified current object.
- *Hint: Remember that the initial memory location of the array is NULL.*

3.6.2.6 `operator+=(const VertexArray&): VertexArray&`

- This operator should append the contents of the passed-in parameter to the current object's array.
- The operator should make use of deep copy.
- Return the modified current object.

3.6.2.7 `operator[](int): Vector&` and `operator[](int): const Vector&`

- This operator should return the `Vector` object at the passed-in position.
- If the position is invalid, throw an object of type `InvalidIndex`.

3.6.2.8 `operator int()`

- This casting operator should return the current size of the array.

3.6.2.9 `operator<<(ostream&, const VertexArray&): ostream&`

- This function should populate the passed-in `ostream` object with the `Vector` objects contained in the array.
- Return the populated `ostream` object.
- *Hint: use the appropriate `Vector` function to populate the `ostream` object.*

3.7 Shape

This class will act as the base class for all shapes that can be drawn by the 2D graphics engine. The `Shape.h` file has been provided for you and should not be modified. In order to reduce the size of the specification, the abstract functions will be discussed here. If a derived class contains the function but does not explain it, please refer to the discussion in this section.

3.7.1 Functions

3.7.1.1 `~Shape()`

- This is an empty destructor.

3.7.1.2 `operator=(const Shape&): Shape&`

- This is a basic assignment operator.

3.7.1.3 `getArray(): VertexArray`

- This function should create a `VertexArray` object containing all the vertices of the shape.

3.7.1.4 `applyMatrix(const TransformationMatrix&): Shape*`

- This function should apply the passed-in transformation matrix to the shape.

3.7.1.5 `clone(): Shape*`

- This function should return a copy of the current object.

3.7.1.6 `formVertices(): VertexArray`

- This function should create a `VertexArray` object containing all the vertices of the shape.
- *Hint: The difference between `getArray` and `formVertices` will become apparent in the derived classes.*

3.7.1.7 `operator+=(const Shape&): ComboShape*`

- This function should create a `ComboShape` by combining the current `Shape` and the passed-in `Shape`.

3.7.1.8 `operator+=(const ComboShape*): ComboShape*`

- This function should create a new `ComboShape` by combining the current `Shape` and the passed-in `ComboShape`.

3.8 ComboShape

A `ComboShape` is a `Shape` that is built out of more than one other shape.

3.8.1 Members

3.8.1.1 `numSubShapes: int`

- This is the number of `Shape`s the `ComboShape` contains.

3.8.1.2 `subShapes: Shape**`

- This is a 1D daynamic array of dynamic `Shape` "objects", which comprises the `ComboShape`.

3.8.2 Functions

3.8.2.1 `ComboShape()`

- The constructor should initialise both member variables to 0/`NULL`.

3.8.2.2 `ComboShape(const ComboShape&)`

- The copy constructor should create a deep copy of the passed-in object.

3.8.2.3 `~ComboShape()`

- The destructor should deallocate all of the dynamic memory assigned to the object.

3.8.2.4 `operator=(const ComboShape&): ComboShape&`

- This assignment operator should assign the passed-in object to the current object as discussed in the lectures.
- Make use of deep copies.

3.8.2.5 `getArray(): VertexArray`

- This function should loop through all of the subshapes and create a combined `VertexArray` which is then returned, using the `formVertices` function.
- If there are no subshapes, return an empty `VertexArray` object.

3.8.2.6 `applyMatrix(const TransformationMatrix&): ComboShape*`

- This function should loop through all of the subshapes and apply the passed-in matrix to each object.
- Add the result of each matrix application to a new `ComboMatrix` which is then returned.

3.8.2.7 `formVertices(): VertexArray`

- This function should return the same result as `getArray`.

3.8.2.8 `operator+=(const Shape&): ComboShape*`

- This function should append the passed-in `Shape` to the sub-shapes array by resizing the array.
- Return the modified current `ComboShape`.

3.8.2.9 `operator+=(const ComboShape*): ComboShape*`

- This function should append all of the sub-shapes in the passed-in `ComboShape` to the current sub-shapes array, by expanding the array.
- Return the modified current `ComboShape`.
- If the passed-in parameter is NULL, do not add it to the sub-shapes array.

3.9 Point

A point is the most primitive shape that can be drawn, and represents a single coordinate in space.

3.9.1 Members

3.9.1.1 `point: Vector`

- The vector represents the position of the point in space.

3.9.2 Functions

3.9.2.1 `Point(const Vector&)`

- This constructor uses the passed-in parameter to populate the member variable.
- The constructor should also set the value at index 2 in the vector to 1.

3.9.2.2 `Point(const Point&)`

- This copy constructor should create a copy of the passed-in point.

3.9.2.3 `operator=(const Point&): Point&`

- This assignment operator should assign the passed-in parameter to the current object as discussed in the lectures.
- Return the modified current object.

3.9.2.4 `getArray(): VertexArray`

- This function should create a `VertexArray` object, add the point to it, and return the `VertexArray` object.

3.9.2.5 `applyMatrix(const TransformationMatrix&): Point*`

- This function should multiply the passed-in matrix with the point, and return a new Point populated with the result.

3.9.2.6 `operator+=(const Shape&): ComboShape*` and `operator+=(const ComboShape*): ComboShape*`

- These functions should return NULL.

3.9.2.7 `getVector(): const Vector&`

- This function should return the point.

3.10 Triangle

A triangle is a shape that contains three points.

3.10.1 Members

3.10.1.1 `p0: Point`, `p1: Point`, and `p2: Point`

- These are the points of the triangle.

3.10.2 Functions

3.10.2.1 `Triangle(const Vector& p0, const Vector& p1, const Vector& p2)`

- This constructor should initialise the three points with the passed-in parameters accordingly.
- Note that the order of the parameters is important, i.e., parameter `p0` should be used to initialise member `p0` and so on.
- If any of the three passed-in parameters have a dimensionality that is not equal to 3, throw an object of type `InvalidVectorSize`.

3.10.2.2 `Triangle(const Triangle&)`

- This copy constructor should create a copy of the passed-in object.

3.10.2.3 `formVertices(): VertexArray`

- Create a `VertexArray`, and populate it as follows:
  1. `p0`
  2. `p1`
  3. `p1`
  4. `p2`
  5. `p2`
  6. `p0`
- Return the populated `VertexArray` object.
- The reason for this order is due to how the screen uses the `VertexArray` to draw lines.

3.10.2.4 `applyMatrix(const TransformationMatrix&): Traingle*`

- This function should apply the passed-in matrix to each point, and using the produced result, create a new triangle and return it.

3.11 Rectangle

This class represents a rectangle shape that is composed of two triangles. Note the following abbreviations:

- `tl` implies top left.
- `tr` implies top right.
- `br` implies bottom right.
- `bl` implies bottom left.

3.11.1 Functions

3.11.1.1 `Rectangle(const Vector& lt, const Vector& tr, const Vector& br, const Vector& bl)`

- This constructor should create two triangles and store them in the subShapes array.
- The triangles should be created as follows:
  1. First triangle: tl, tr, br
  2. Second triangle: tl, bl, br

3.11.1.2 `Rectangle(const Rectangle&)`

- This copy constructor should create a deep copy of the passed-in parameter.

3.11.1.3 `formVertices(): VertexArray`

- This function should create a `VertexArray` object, and first populate it with the result of the first triangle's `formVertices` function.
- Next, the function should append the object with the result of the second triangle's `formVertices` function.
- Return this created object.

3.12 Screen

The screen is what will be used to draw the different shapes to the terminal. The process is first to obtain a shape's `VertexArray`, then, in groups of two, draw lines. The problem is that the screen works in integers while positions of points are floats. We will be using the Midpoint Line Generation Algorithm[2].

3.12.1 Members

3.12.1.1 `pixels: char**`

- This is a dynamic 2D array that represents the individual pixels of the "screen".

3.12.1.2 `height: int`

- This is the height(h) of the screen.

3.12.1.3 `width: int`

- This is the width(w) of the screen.

Thus, a screen is a $h \times w$ 2D array of pixels.

3.12.2 Functions

3.12.2.1 `Screen(height: int, width: int)`

- The constructor should initialise all the member variables appropriately.
- Populate the pixels array with space characters (`' '`).

3.12.2.2 `~Screen()`

- The destructor should deallocate all the memory assigned to the object.

[2]More information about the Midpoint algorithm can be found at: `https://www.geeksforgeeks.org/dsa/mid-point-line-generation-algorithm/`. Note: do not simply copy and paste the C++ code from the resource.

3.12.2.3 `operator<<(ostream&, const Screen&): ostream&`

- This operator should populate the `ostream` object with the pixel array.
- The operator also needs to place a border around the screen, using the hash (`'#'`) character.
- For example, assume a 5×4 screen; the `ostream` object should be populated as follows:

```
######
#    #
#    #
#    #
#    #
#    #
######
```

- Please see the example output for more examples.

3.12.2.4 `implicitLineEquation(const Vector& p0, const Vector& p1, const Vector& p): float`

- This function assumes that index 0 of a vector is seen as the x-coordinate of a point, and that index 1 of a vector is seen as the y-coordinate of a point.
- This function calculates the following equation:

$$f(p_0, p_1, p) = p_x(p_{0,y} - p_{1,y}) + p_{1,y}(p_{0,x} - p_{1,x}) - (p_{1,x} * p_{0,y}) \tag{1}$$

- *Hint:* $p_{0,y}$ *refers to the* y*-component of point* p_0

3.12.2.5 `draw(int x, int y): void`

- This function should mark the specified position in the pixel array with an X.
- Due to the orientation of the array compared to the screen, a conversion mapping is followed: $(x, y) \mapsto (x, h - y - 1)$[3].
- If the passed-in coordinates are outside the bounds of the screen, simply return.
- *Hint: Remember that* x *is the related to the width and* y *is related to the height of the pixel array.*

3.12.2.6 `draw(const Vector& p0, const Vector& p1): void`

- This function should draw a line from point `p0` to `p1` using the Mid Point algorithm as described by Algorithm A.1.
- Note that you need to ensure that any time you use `p0` or `p1` in Algorithm A.1, you need to first round the vectors.

[3]The symbol $\mapsto$ denotes a mapping.

- It is recommended that you add non-member helper functions to break up the algorithm into smaller reusable parts.

3.12.2.7 `draw(const VertexArray&): void`

- This function should take every two pairs of vectors in the vertex arrays and pass them to the appropriate `draw` function.
- In other words, if the `VertexArray` object contains the vectors: $v0, v1, v2, v3, v4, v5, v6$, then the `draw` calls should be:
  1. `draw(v0, v1)`
  2. `draw(v2, v3)`
  3. `draw(v4, v5)`

3.12.2.8 `clearScreen(): void`

- This function needs to reset each character in the pixels array to space characters.

4 Memory Management

As memory management is a core part of C++ programming, each task on FitchFork will allocate approximately 10% of the marks to memory management. The following command is used:

```
valgrind --leak-check=full ./main
```

Please ensure, at all times, that your code *correctly* de-allocates *all* the memory that was allocated.

5 Testing

As testing is a vital skill that all software developers need to know and be able to perform daily, 10% of the assignment marks will be allocated to your testing skills. To do this, you will need to submit a testing main (inside the `main.cpp` file) that will be used to test an Instructor Provided solution. You may add any helper functions to the main.cpp file to aid your testing. In order to determine the coverage of your testing the gcov [4] tool, specifically the following version *gcov (Debian 8.3.0-6) 8.3.0*, will be used. The following set of commands will be used to run gcov:

```
g++ --coverage *.cpp -o main
./main
gcov -f -m -r -j ${files}
```

This will generate output which we will use to determine your testing coverage. The following coverage ratio will be used:

$$\frac{\text{number of lines executed}}{\text{number of source code lines}}$$

[4] For more information on gcov please see `https://gcc.gnu.org/onlinedocs/gcc/Gcov.html`

We will scale this ration based on class size.

The mark you will receive for the testing coverage task is determined using Table 1:

Coverage ratio range	% of testing mark
0%-5%	0%
5%-20%	20%
20%-40%	40%
40%-60%	60%
60%-80%	80%
80%-100%	100%

Table 1: Mark assignment for testing

Note the top boundary for the Coverage ratio range is not inclusive except for 100%. Also, note that only the functions stipulated in this specification will be considered to determine your testing mark. Remember that your main will be testing the Instructor Provided code and as such, it can only be assumed that the functions outlined in this specification are defined and implemented.

As you will be receiving marks for your testing main, plagiarism detection will also be performed on the testing main.

6 Implementation Details

- You must implement the functions in the header files exactly as stipulated in this specification. Failure to do so will result in compilation errors on FitchFork.
- You may only use **c++98**.
- You may only utilise the specified libraries. Failure to do so will result in compilation errors on FitchFork.
- You may only use the following libraries:
  - iostream
  - cmath
  - iomanip
- You are supplied with a **trivial** main demonstrating the basic functionality of this assessment.

7 Upload Checklist

The following C++ files should be in a zip archive named uXXXXXXXX.zip where XXXXXXXX is your student number:

- ComboShape.cpp
- Matrix.cpp
- Point.cpp
- Rectangle.cpp
- Screen.cpp
- SquareMatrix.cpp
- TransformationMatrix.cpp
- Triangle.cpp
- Vector.cpp
- VertexArray.cpp
- main.cpp

The files should be in the root directory of your zip file. In other words, when you open your zip file you should immediately see your files. They should not be inside another folder.

8 Submission

You need to submit your source file(s), as a single archive, on the FitchFork website (`https://ff.cs.up.ac.za/`). All methods need to be implemented (or at least stubbed) before submission. Your code should be able to be compiled with the following command:

```
g++ -std=c++98 -Werror -Wall *.cpp -o main
```

and run with the following command:

```
./main
```

Remember your `h` file will be overwritten, so ensure you do not alter the provided `h` files.
You have 5 submissions, and your best mark will be your final mark. Upload your archive to the Assignment 2 slot on the FitchFork website. If you submit after the deadline but before the late deadline, a 20% mark deduction will be applied. **No submissions after the late deadline will be accepted!**

A Midpoint Algorithm

Algorithm A.1 Draw Line using Midpoint Algorithm

```
1:  function DRAW(p0, p1, screen)
2:      if p0 > p1 then
3:          Draw(p1, p0, screen)                          ▷ swap endpoints if needed
4:          return
5:      end if
6:      if p0[0] == p1[0] then
7:          yi ← min(p0[1], p1[1])                        ▷ vertical line lower bound
8:          yf ← max(p0[1], p1[1])                        ▷ vertical line upper bound
9:          for y = yi to yf do
10:             screen.draw(p0[0], y)                     ▷ plot vertical pixel
11:         end for
12:     else if p0[1] == p1[1] then
13:         xi ← min(p0[0], p1[0])                        ▷ horizontal line left bound
14:         xf ← max(p0[0], p1[0])                        ▷ horizontal line right bound
15:         for x = xi to xf do
16:             screen.draw(x, p1[1])                     ▷ plot horizontal pixel
17:         end for
18:     else
19:         m ← gradient(p0, p1)                          ▷ compute slope
20:         if 0 < m < 1 then
21:             y ← p0[1]                                 ▷ gentle positive slope
22:             for x = p0[0] to p1[0] do
23:                 screen.draw(x, y)
24:                 v ← (x+1, y+0.5)
25:                 if implicitLineEquation(p0, p1, v) < 0 then
26:                     y ← y + 1                         ▷ increment y if below line
27:                 end if
28:             end for
29:         else if -1 < m < 0 then
30:             y ← p0[1]                                 ▷ gentle negative slope
31:             for x = p0[0] to p1[0] do
32:                 screen.draw(x, y)
33:                 v ← (x+1, y-0.5)
34:                 if implicitLineEquation(p0, p1, v) > 0 then
35:                     y ← y - 1                         ▷ decrement y if above line
36:                 end if
37:             end for
38:         else if m ≥ 1 then
39:             x ← p0[0]                                 ▷ steep positive slope
40:             for y = p0[1] to p1[1] do
41:                 screen.draw(x, y)
42:                 v ← (x+0.5, y+1)
43:                 if implicitLineEquation(p0, p1, v) > 0 then
44:                     x ← x + 1                         ▷ increment x if right of line
45:                 end if
46:             end for
47:         else if m ≤ -1 then
48:             x ← p0[0]                                 ▷ steep negative slope
49:             for y = p0[1] downto p1[1] do
50:                 screen.draw(x, y)
51:                 v ← (x+0.5, y-1)
52:                 if implicitLineEquation(p0, p1, v) < 0 then
53:                     x ← x + 1                         ▷ increment x if left of line
54:                 end if
55:             end for
56:         end if
57:     end if
58: end function
```

B Background

B.1 Implicit Line Equation

A line can be described in various forms. One common form is the *slope-intercept form*:

$$y = mx + c$$

where:

- m is the slope of the line,
- c is the y-intercept.

However, another useful way to define a line is by specifying two points on the line:

$$P_1 = (x_1, y_1), \quad P_2 = (x_2, y_2)$$

Using these two points, we can compute the slope m as:

$$m = \frac{y_2 - y_1}{x_2 - x_1} \quad \text{(provided } x_2 \neq x_1\text{)}$$

Substituting one of the points (say, P_1) into the slope-intercept form gives:

$$y - y_1 = m(x - x_1)$$

Expanding and rearranging:

$$y - mx + (mx_1 - y_1) = 0$$

This gives us the *implicit form* of the line:

$$Ax + By + C = 0$$

where:

$$A = -(y_2 - y_1), \quad B = x_2 - x_1, \quad C = (y_1(x_2 - x_1)) - (x_1(y_2 - y_1))$$

Putting it all together, we can obtain the following equation:

$$f(x, y) \equiv (y_0 - y_1)x + (x_1 - x_0)y + (x_0y_1 - x_1y_0) = 0 \tag{2}$$

B.2 Computer Graphics Screen

A simple terminal-based graphics engine is a lightweight software library or tool that allows the creation of visual representations, like shapes, animations, or basic games, within a terminal or command-line interface. Instead of using pixels, it typically uses characters (like #, *, or ASCII symbols) to simulate graphics. These engines handle screen refreshing, input handling, and coordinate-based drawing using simple logic and terminal control codes (like ANSI escape sequences). They're often used for retro-style visuals, learning graphics fundamentals, or quick prototyping in environments without GUI support.

A "screen" is defined with coordinate $(0, 0)$ being the origin and at the bottom left of the screen. The maximum number of rows that the screen can have is denoted by n_y, and the maximum number of columns that the screen can have is denoted by n_x. Given that $n_x = 10$ and $n_y = 6$, Figure 2 shows how the screen would be laid out, where each cell represents a pixel:

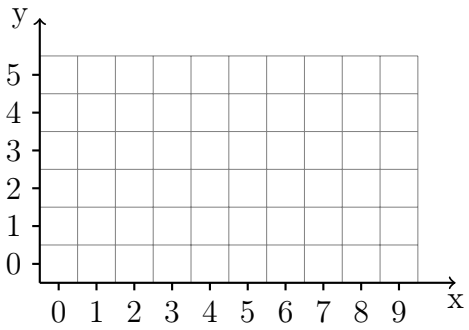


Figure 2: Example of a screen layout with $n_x = 10$ and $n_y = 6$.

B.3 Mid Point Algorithm

A big problem in computer graphics is how to draw a straight line when the line does not perfectly go over the centre of a pixel. Consider Figure 3, which contains two lines we would like to draw.

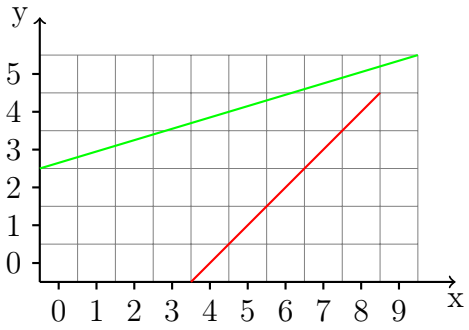


Figure 3: Example of a screen layout with $n_x = 10$ and $n_y = 6$, showing two lines.

When drawing the red line in our terminal screen, we can easily "colour" in the cells that the red line passes through as seen in Figure 4.

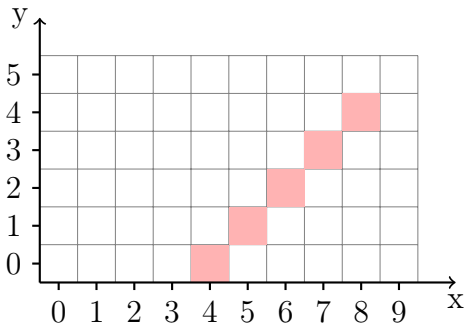


Figure 4: Screen layout with $n_x = 10$ and $n_y = 6$, showing how the pixels will be coloured for the red line.

The green line, on the other hand, is not as easy. This is where the midpoint line drawing algorithm comes into effect. The idea of the midpoint algorithm is that you use the implicit line equation of the line you are trying to draw, to get a "reasonable" resemblance of the line. An example of the results obtained from the mid-point algorithm for the green line is given in Figure 5.

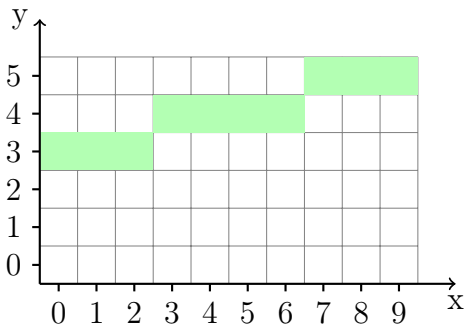


Figure 5: Screen layout with $n_x = 10$ and $n_y = 6$, showing how the pixels will be coloured for the green line, using the mid point algorithm.